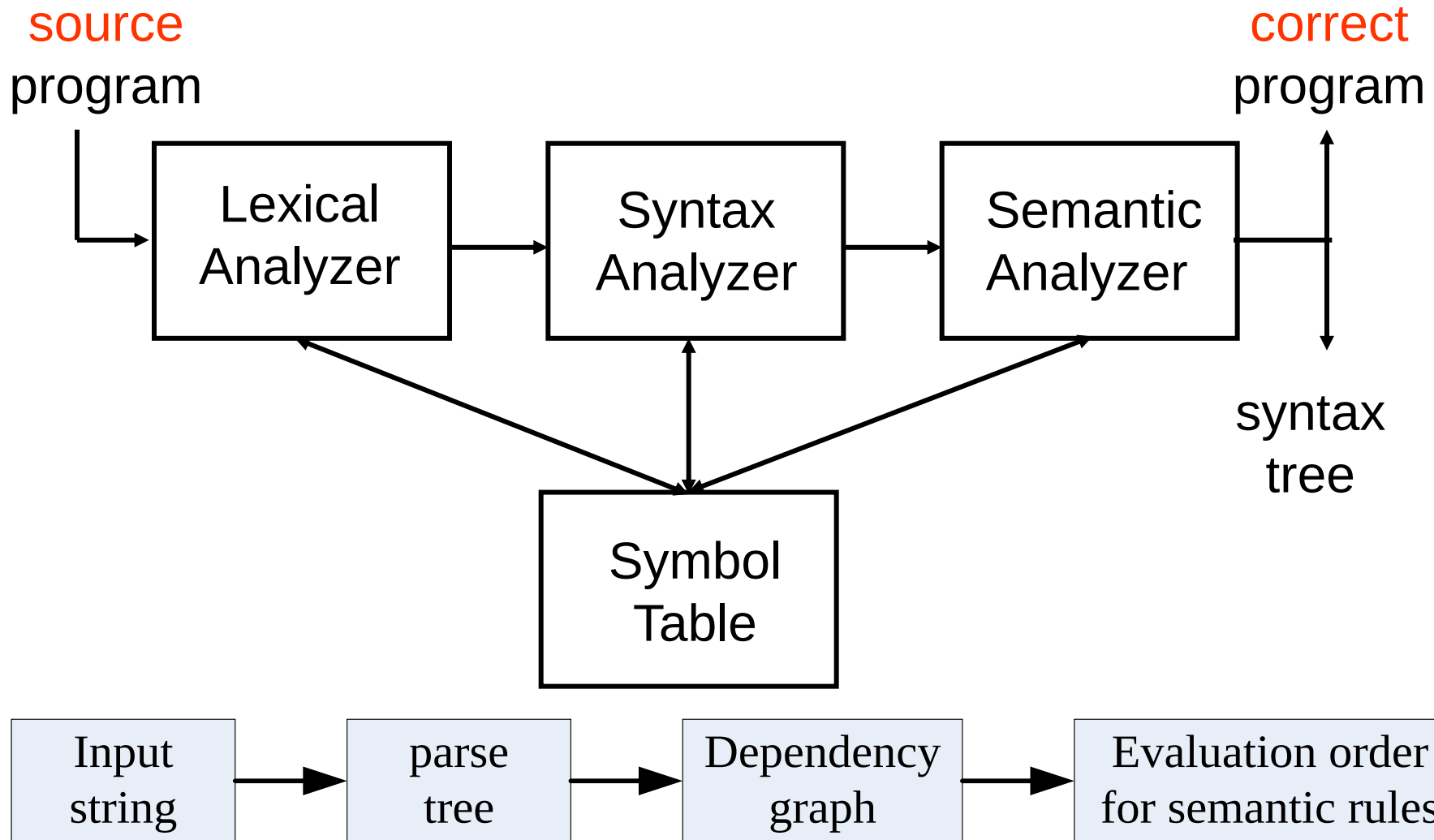


Semantic Analysis

◆ Semantic Analyzer

- Attribute Grammars
- Syntax Tree Construction
- Top-Down Translators
- Recursive Evaluators
- Bottom-Up Translators
- Type Checking (next chapter)

Semantic Analyzer





Syntax-Directed Translation 语法制导翻译

- ◆ Grammar symbols are associated with **attributes** to associate information with the programming language constructs that they represent.
- ◆ Values of these attributes are evaluated by the **semantic rules** （语义规则） associated with the production rules.
- ◆ Evaluation of these semantic rules:
 - may generate intermediate codes
 - may put information into the symbol table
 - may perform type checking
 - may issue error messages
 - may perform some other activities
 - in fact, they may perform almost any activities.
- ◆ An attribute may hold almost any thing.
 - ◆ a string, a number, a memory location, a complex record.

Syntax-Directed Definitions and Translation Schemes

- ◆ When we associate semantic rules with productions, we use two notations:
 - ◆ Syntax-Directed Definitions
 - ◆ Translation Schemes
- ◆ Syntax-Directed Definitions:
 - ◆ give high-level specifications for translations
 - ◆ hide many implementation details such as order of evaluation of semantic actions.
 - ◆ We associate a production rule with a set of semantic actions, and we do not say *when* they will be evaluated.
- ◆ Translation Schemes:
 - ◆ indicate the order of evaluation of semantic actions associated with a production rule.
 - ◆ In other words, translation schemes give a little bit information about implementation details.

Syntax-Directed Definitions

- ◆ A syntax-directed definition is a generalization of a context-free grammar in which:
 - ◆ Each grammar symbol is associated with a set of attributes.
 - ◆ This set of attributes for a grammar symbol is partitioned into two subsets called **synthesized** and **inherited attributes** of that grammar symbol.
 - ◆ Each production rule is associated with a set of semantic rules.
- ◆ **Semantic rules** set up dependencies between attributes which can be represented by a *dependency graph*.
- ◆ This *dependency graph* determines the evaluation order of these semantic rules.
- ◆ Evaluation of a semantic rule defines the value of an attribute. But a semantic rule may also have some side effects such as printing a value.

Form of a Syntax-Directed Definition (Semantic Rules)

In a syntax-directed definition, each production $A \rightarrow \alpha$ is associated with a set of semantic rules of the form:

$b = f(c_1, c_2, \dots, c_n)$ where f is a function,

and b can be one of the followings:

→ b is a **synthesized attribute** of A and $c_1, c_2, \dots, c_n$ are attributes of the grammar symbols in the production ($A \rightarrow \alpha$).

OR

→ b is an **inherited attribute** of one of the grammar symbols in α (on the right side of the production), and $c_1, c_2, \dots, c_n$ are attributes of the grammar symbols in the production ($A \rightarrow \alpha$).

Attribute Grammars

- ◆ Each **grammar symbol** is associated with a set of semantic attributes
- ◆ Each **production** is associated with a set of semantic rules for computing semantic attributes
- ◆ An **attribute grammar** is a context free grammar with associated semantic attributes and semantic rules

Attribute Grammar (cont.)

- ◆ So, a semantic rule $b=f(c_1, c_2, \dots, c_n)$ indicates that the attribute b *depends on* attributes $c_1, c_2, \dots, c_n$.
- ◆ In a **syntax-directed definition**, a semantic rule may just evaluate a value of an attribute or it may have some side effects such as printing values.
- ◆ An **attribute grammar** is a syntax-directed definition in which the functions in the semantic rules cannot have side effects (they can only evaluate values of attributes).

Syntax-Directed Definition – Example 5.1



Production

$L \rightarrow E \text{ return}$

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

Semantic Rules

$\text{print}(E.\text{val})$

$E.\text{val} = E_1.\text{val} + T.\text{val}$

$E.\text{val} = T.\text{val}$

$T.\text{val} = T_1.\text{val} * F.\text{val}$

$T.\text{val} = F.\text{val}$

$F.\text{val} = E.\text{val}$

$F.\text{val} = \text{digit}.\text{lexval}$

- Symbols E , T , and F are associated with a synthesized attribute *val*.
- The token *digit* has a synthesized attribute *lexval* (it is assumed that it is evaluated by the lexical analyzer).

Annotated Parse-Trees

- ◆ Parse-tree that also shows the values of the attributes at each node.
- ◆ Attributes in the leaves of the annotated parse-tree are determined by the lexical analyzer.
- ◆ Values of attributes in inner nodes of annotated parse-tree are determined by the semantic-rules.
- ◆ If a syntax-directed definition employs only *Synthesized* attributes the evaluation of all attributes can be done in a bottom-up fashion.
- ◆ *Inherited* attributes would require more arbitrary “traversals” of the annotated parse-tree.
- ◆ A *dependency graph* suggests possible *evaluation orders* for an annotated parse-tree.

Annotated Parse Tree – Example

Input: 5+3*4

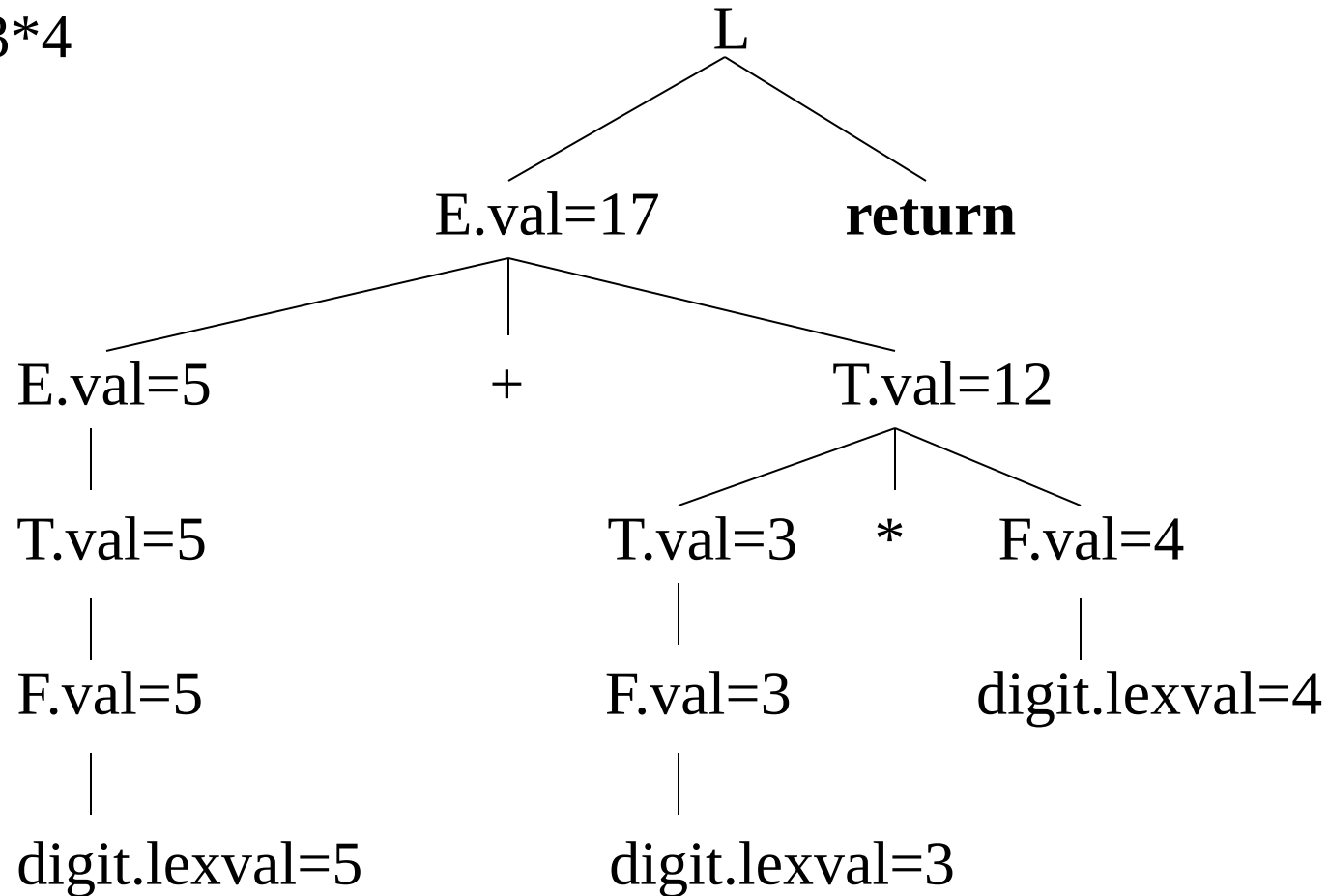


Fig. 5.3. Annotated parse tree for 5+3*4

Synthesized Attributes

$L \rightarrow E$ return

print(E.val)

$E \rightarrow E_1 + T$

$E.val := E_1.val + T.val$

$E \rightarrow T$

$E.val := T.val$

$T \rightarrow T_1 * F$

$T.val := T_1.val * F.val$

$T \rightarrow F$

$T.val := F.val$

$F \rightarrow (E)$

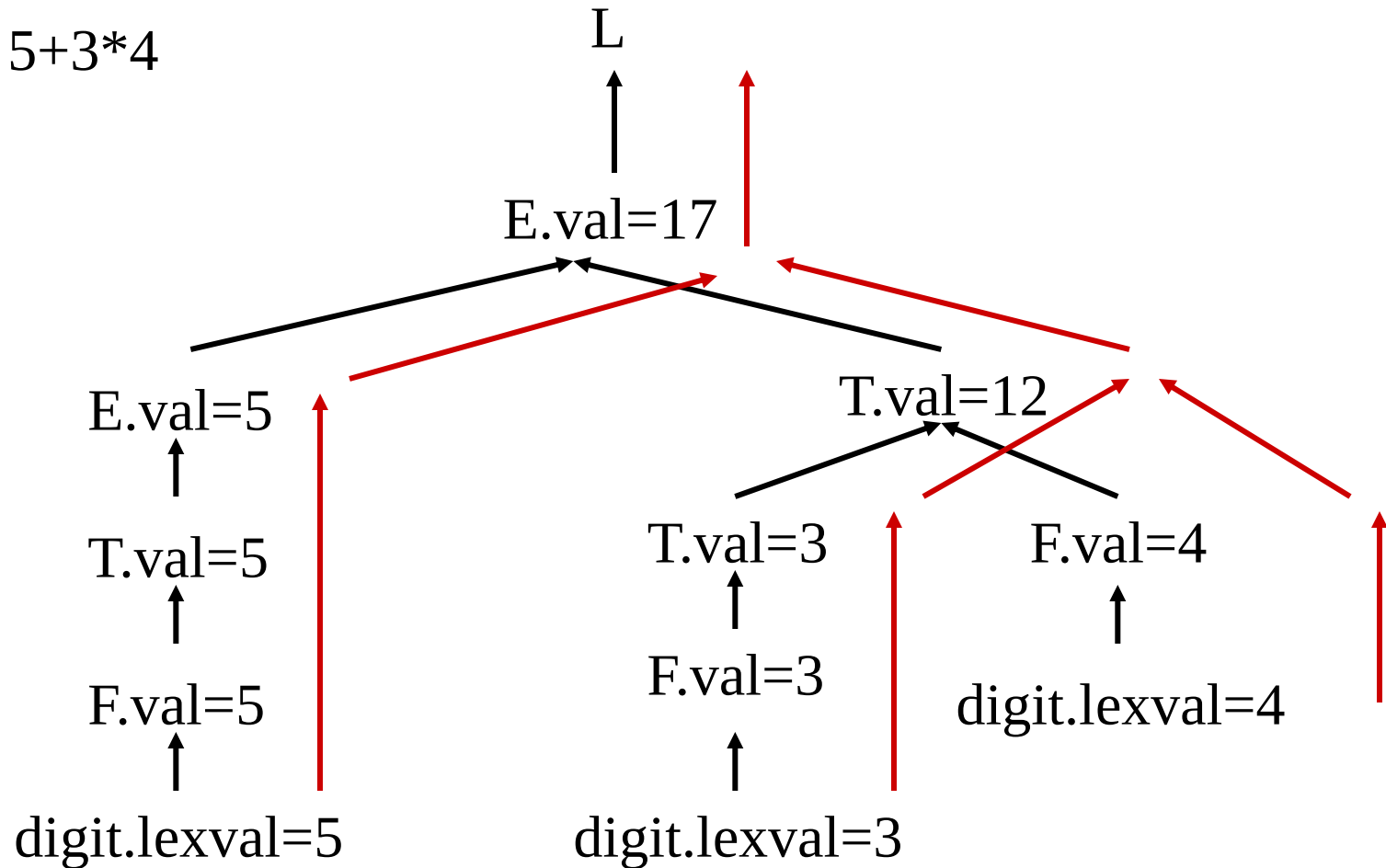
$F.val := E.val$

$F \rightarrow \mathbf{digit}$

$F.val := \mathbf{digit.lexval}$

Synthesized attributes

Input: $5+3*4$



S-attributed definition can be annotated by evaluating the semantic rules for the attributes at each node bottom up, from the leaves to the root.

Inherited Attributes

Production

$D \rightarrow T L$

$T \rightarrow \text{int}$

$T \rightarrow \text{real}$

$L \rightarrow L_1, \text{id}$

$L \rightarrow \text{id}$

Semantic Rules

$L.in = T.type$

$T.type = \text{integer}$

$T.type = \text{real}$

$L_1.in = L.in, \text{ addtype}(\text{id.entry}, L.in)$

$\text{addtype}(\text{id.entry}, L.in)$

- Symbol T is associated with a synthesized attribute *type*.
- Symbol L is associated with an inherited attribute *in*.

Inherited Attributes (cont.)

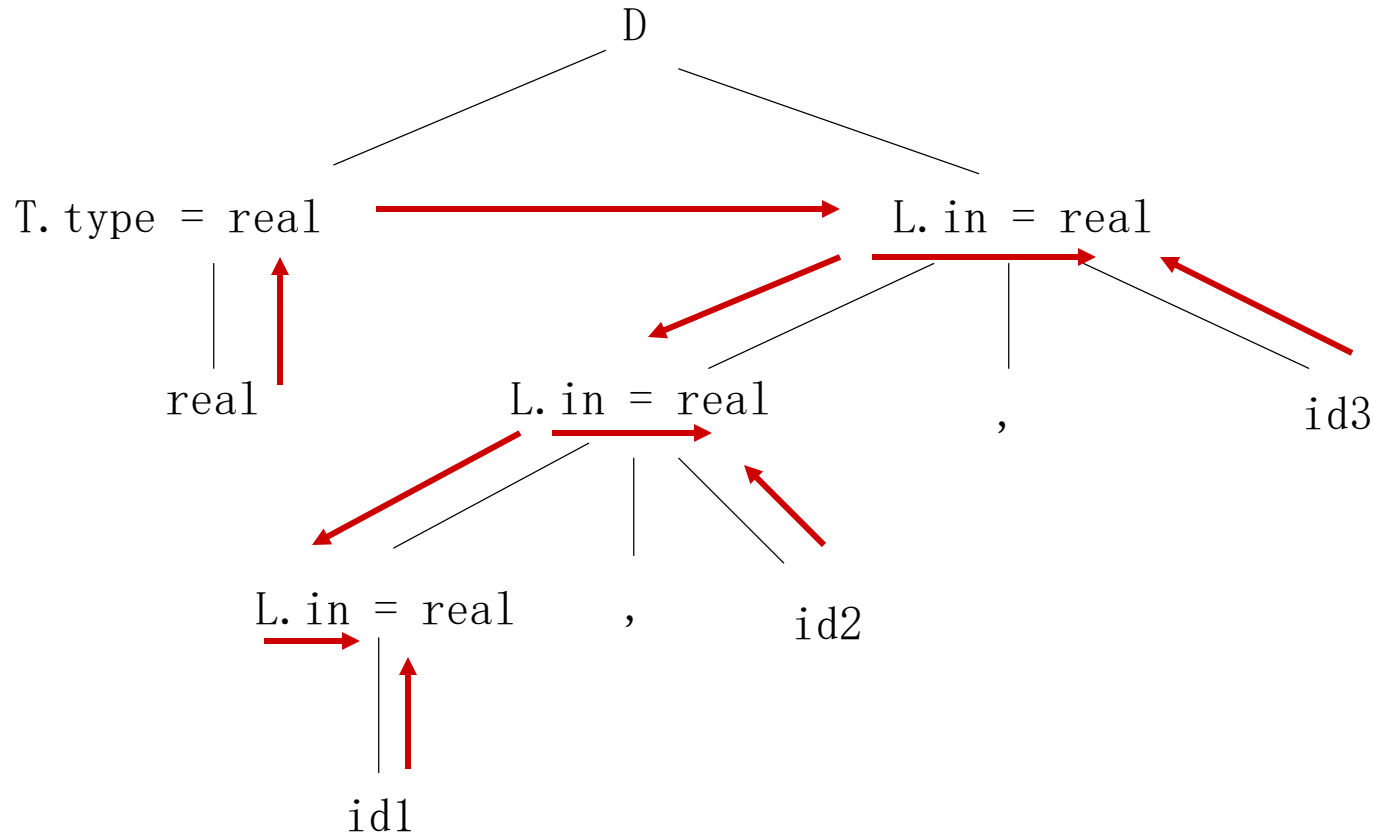


Fig 5.5 parse tree with inherited attribute in at each node labeled L (P.284)

Dependencies of Attributes

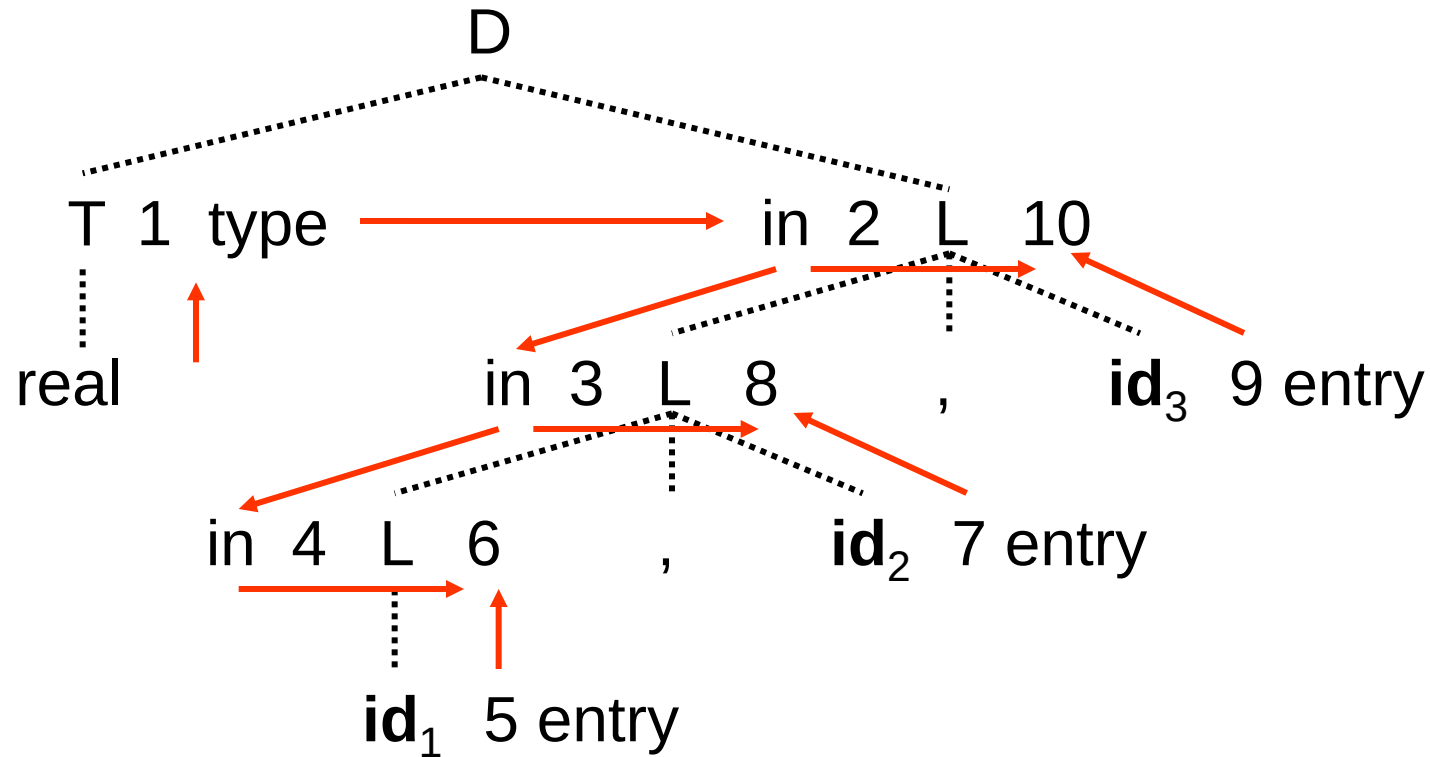
- ◆ In the semantic rule

$$b := f(c_1, c_2, \dots, c_k)$$

we say b **depends on** $c_1, c_2, \dots, c_k$

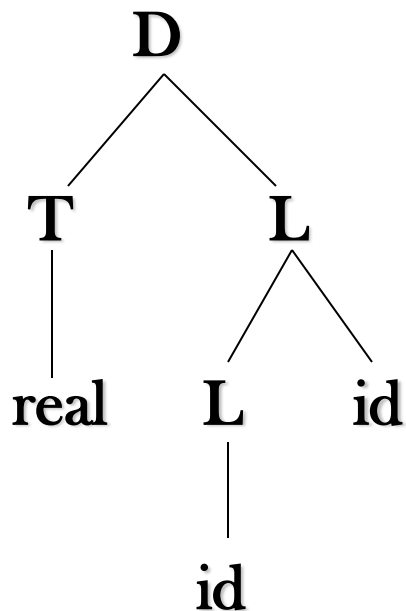
- ◆ The semantic rule for b must be evaluated **after** the semantic rules for $c_1, c_2, \dots, c_k$
- ◆ The dependencies of attributes can be represented by a directed graph called **dependency graph**

Dependency Graphs - Inherited Attributes

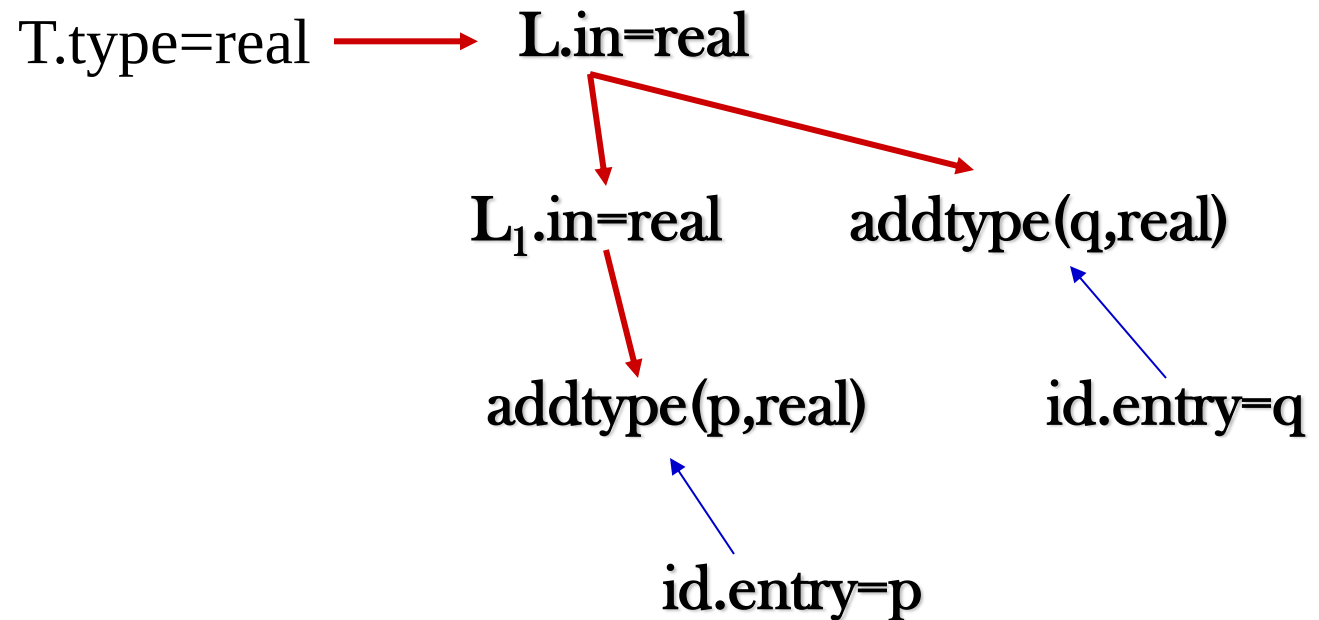


Dependency Graph (cont.)

Input: real p q



parse tree



dependency graph

S-Attributed Definitions

L-Attributed Definitions

- ◆ Syntax-directed definitions are used to specify syntax-directed translations.
- ◆ To create a translator for an arbitrary syntax-directed definition can be difficult.
- ◆ We would like to evaluate the semantic rules during parsing (i.e. in a single pass, we will parse and we will also evaluate semantic rules during the parsing).
- ◆ We will look at two sub-classes of the syntax-directed definitions:
 - ◆ **S-Attributed Definitions**: only synthesized attributes used in the syntax-directed definitions.
 - ◆ **L-Attributed Definitions**: in addition to synthesized attributes, we may also use inherited attributes in a restricted fashion.
- ◆ To implement S-Attributed Definitions and L-Attributed Definitions are easy (we can evaluate semantic rules in a single pass during the parsing).
- ◆ Implementations of S-attributed Definitions are a little bit easier than implementations of L-Attributed Definitions.

S-Attributed Attribute Grammars

- An attribute grammar is **S-attributed** if it uses synthesized attributes exclusively

An example:

$L \rightarrow E$	return	print(E.val)
$E \rightarrow E_1 + T$		$E.val := E_1.val + T.val$
$E \rightarrow T$		$E.val := T.val$
$T \rightarrow T_1 * F$		$T.val := T_1.val * F.val$
$T \rightarrow F$		$T.val := F.val$
$F \rightarrow (E)$		$F.val := E.val$
$F \rightarrow \text{digit}$		$F.val := \text{digit.lexval}$

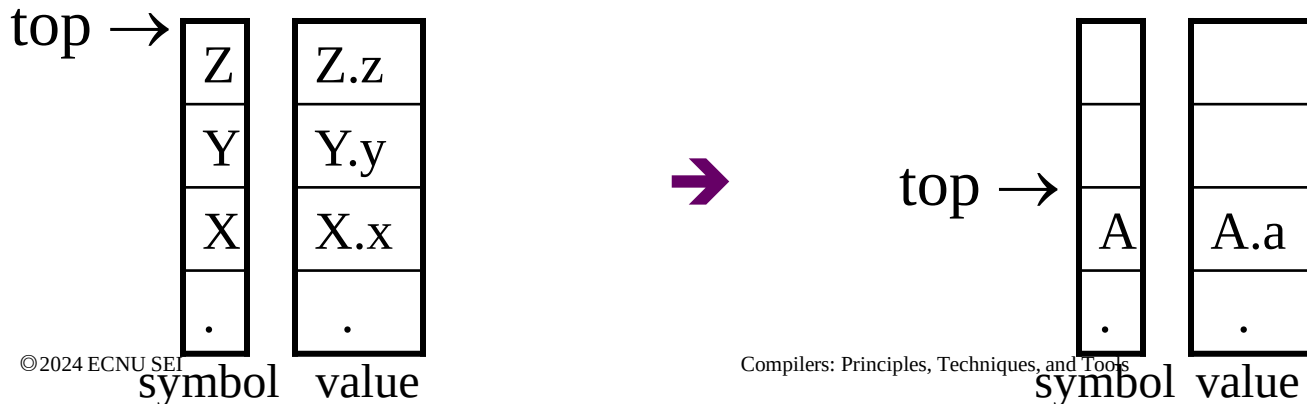
Bottom-Up Evaluation of S-Attributed Definitions



- ◆ S-attributed Definition: Syntax-Directed Definition using only Synthesized attributes.
- ◆ Stack of a LR(1) parser contains states.
- ◆ Recall that each state corresponds to some grammar symbol (and many different states might correspond to the same grammar symbol)
- ◆ Extend stack entries to include the attribute(s).
- ◆ Modify stack contents as productions are selected.

$A \rightarrow XYZ$ $A.a = f(X.x, Y.y, Z.z)$ where all attributes are synthesized.

stack parallel-stack



Bottom-Up Eval. of S-Attributed Definitions (cont.)

Production

$L \rightarrow E \text{ return}$

$E \rightarrow E1 + T$

$E \rightarrow T$

$T \rightarrow T1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

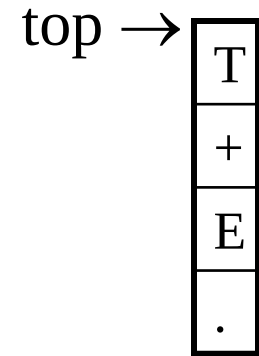
Semantic Rules

$\text{print}(\text{val}[\text{top}-1])$

$\text{val}[\text{ntop}] = \text{val}[\text{top}-2] + \text{val}[\text{top}]$

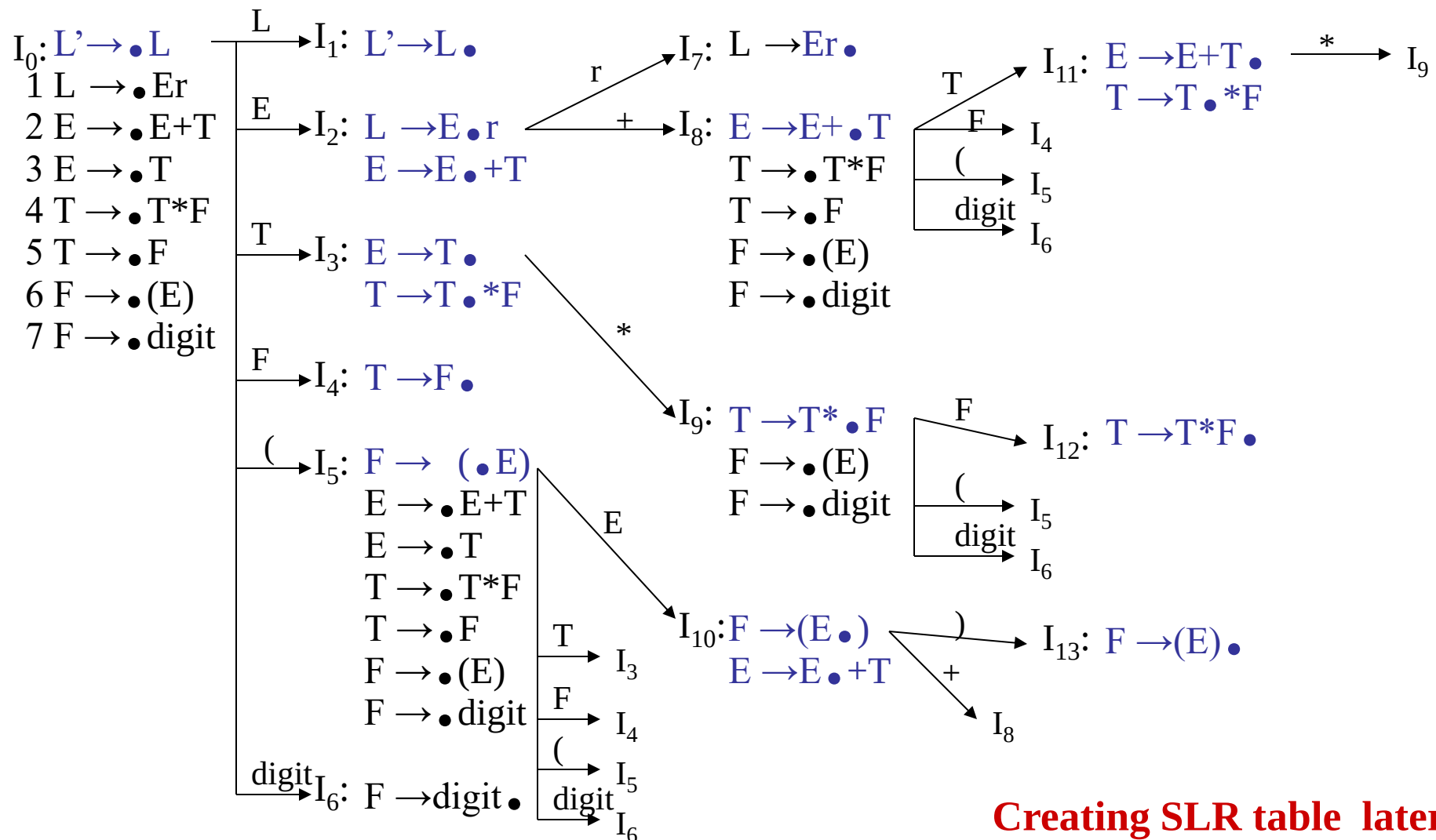
$\text{val}[\text{ntop}] = \text{val}[\text{top}-2] * \text{val}[\text{top}]$

$\text{val}[\text{ntop}] = \text{val}[\text{top}-1]$



- At each shift of digit, we also push digit.lexval into *val-stack*.
- At all other shifts, we do not put anything into *val-stack* because other terminals do not have attributes (but we increment the stack pointer for *val-stack*).

Canonical LR(0) Collection for The Grammar



Creating SLR table later

SLR(1) parsing table

states	Action table							Goto table			
	+	*	(	)	digit	r	\$	L	E	T	F
0			S5		S6			1	2	3	4
1							acc				
2	S8					S7					
3	R3	S9		R3							
4	R5	R5		R5							
5			S5		S6				10	3	4
6	R7	R7		R7							
7							R1				
8			S5		S6					11	4
9			S5		S6						12
10	S8			S13							
11	R2	S9		R2							
12	R4	R4		R4							
13	R6	R6		R6							

Bottom-Up Evaluation – Example 5.10



- At each shift of digit, we also push `digit.lexval` into *val-stack*.

<u>stack</u>	<u>val-stack</u>	<u>input</u>	<u>action</u>	<u>semantic rule</u>
0		5+3*4r	s6	digit.lexval(5) into val-stack
0digit6	5	+3*4r	F→digit	F.val=d.lexval – do nothing
0F4	5	+3*4r	T→F	T.val=F.val – do nothing
0T3	5	+3*4r	E→T	E.val=T.val – do nothing
0E2	5	+3*4r	s8	push empty slot into val-stack
0E2+8	5-	3*4r	s6	digit.lexval(3) into val-stack
0E2+8digit6	5-3	*4r	F→digit	F.val=digit.lexval – do nothing
0E2+8F4	5-3	*4r	T→F	T.val=F.val – do nothing
0E2+8T11	5-3	*4r	s9	push empty slot into val-stack
0E2+8T11*9	5-3-	4r	s6	digit.lexval(4) into val-stack
0E2+8T11*9digit6	5-3-4	r	F→digit	F.val=digit.lexval – do nothing
0E2+8T11*9F12	5-3-4	r	T→T*F	T.val=T ₁ .val * F.val
0E2+8T11	5-12	r	E→E+T	E.val=E ₁ .val + T.val
0E2	17	r	s7	push empty slot into val-stack
0E2r7	17-	\$	L→Er	print(17), pop empty slot from val-stack
0L1	17	\$	acc	

L-Attributed Definitions (P.296)

- ◆ S-Attributed Definitions can be efficiently implemented.
- ◆ We are looking for a larger (larger than S-Attributed Definitions) subset of syntax-directed definitions which can be efficiently evaluated.
- ◆ ➔ L-Attributed Definitions
- ◆ L-Attributed Definitions can always be evaluated by the depth first visit of the parse tree.
- ◆ This means that they can also be evaluated during the parsing.

L-Attributed Attribute Grammars

- An attribute grammar is **L-attributed** if each attribute computed in each semantic rule for each production

$$A \rightarrow X_1 X_2 \dots X_{j-1} X_j \dots X_n$$

is a **synthesized** attribute, or an **inherited** attribute of X_j , $1 \leq j \leq n$, depending only on

1. the attributes of $X_1, X_2, \dots, X_{j-1}$

2. the inherited attributes of A

An Example

$D \rightarrow T L$

$T \rightarrow \text{int}$

$T \rightarrow \text{float}$

$L \rightarrow L_1, \text{id}$

$L \rightarrow \text{id}$

$L.\text{in} := T.\text{type}$

$T.\text{type} := \text{integer}$

$T.\text{type} := \text{float}$

$L_1.\text{in} := L.\text{in};$

$\text{addtype}(\text{id}.\text{entry}, L.\text{in})$

$\text{addtype}(\text{id}.\text{entry}, L.\text{in})$

A Definition which is NOT L-Attributed

Productions

$A \rightarrow L M$

$A \rightarrow Q R$

Semantic Rules

$L.in = l(A.i), M.in = m(L.s), A.s = f(M.s)$

$R.in = r(A.in), Q.in = q(R.s), A.s = f(Q.s)$

- This syntax-directed definition is not L-attributed because the semantic rule $Q.in = q(R.s)$ violates the restrictions of L-attributed definitions.
- When $Q.in$ must be evaluated before we enter to Q because it is an inherited attribute.
- But the value of $Q.in$ depends on $R.s$ which will be available after we return from R . So, we are not be able to evaluate the value of $Q.in$ before we enter to Q .